

# **Processamento de Linguagens – Trabalho Prático**

Construção de um compilador para Fortran 77

André Dinis Alves Santos   a106854  
Daniel Gonçalves Parente   a107363  
Pedro Francisco Ferreira   a107292

Ano Letivo 2025/2026

## **Conteúdo**

|                                        |           |
|----------------------------------------|-----------|
| <b>1. Introdução</b>                   | <b>2</b>  |
| <b>2. Arquitetura</b>                  | <b>2</b>  |
| <b>3. Pré-Processamento</b>            | <b>3</b>  |
| <b>4. Análise Léxica</b>               | <b>4</b>  |
| <b>5. Análise Sintática</b>            | <b>5</b>  |
| <b>6. Análise Semântica</b>            | <b>7</b>  |
| <b>7. Tradução e Geração de Código</b> | <b>8</b>  |
| <b>8. Testes</b>                       | <b>10</b> |
| <b>9. Manual de Utilização</b>         | <b>11</b> |
| <b>10. Dificuldades Encontradas</b>    | <b>12</b> |
| <b>11. Conclusão</b>                   | <b>13</b> |



## 1. Introdução

O presente documento descreve o desenvolvimento de um compilador para a linguagem Fortran 77, no âmbito da unidade curricular de Processamento de Linguagens do ano letivo 2025/2026.

O compilador foi implementado em Python, recorrendo à biblioteca `PLY` (*Python Lex-Yacc*) para a análise léxica e sintática. O sistema traduz código-fonte Fortran 77 diretamente para código da máquina virtual disponibilizada no contexto da unidade curricular, sem recorrer a uma representação intermédia.

O compilador está organizado em quatro fases sequenciais: análise léxica, análise sintática, análise semântica e geração de código. Cada fase é implementada num módulo Python independente, o que facilita a manutenção e a extensão do sistema.

As funcionalidades implementadas cobrem integralmente os requisitos mínimos do enunciado, incluindo declaração de tipos e variáveis (escalares e arrays), expressões aritméticas, lógicas e relacionais, estruturas de controlo de fluxo (`IF-THEN-ELSEIF-ELSE`, ciclos `DO`, `GOTO`), operações de I/O básicas (`READ`, `PRINT`) e, também a definição e chamada de subprogramas (`FUNCTION` e `SUBROUTINE`).

## 2. Arquitetura

O compilador segue uma arquitetura de *pipeline* linear, onde a saída de cada fase é a entrada da seguinte. A figura 1 ilustra o fluxo completo.

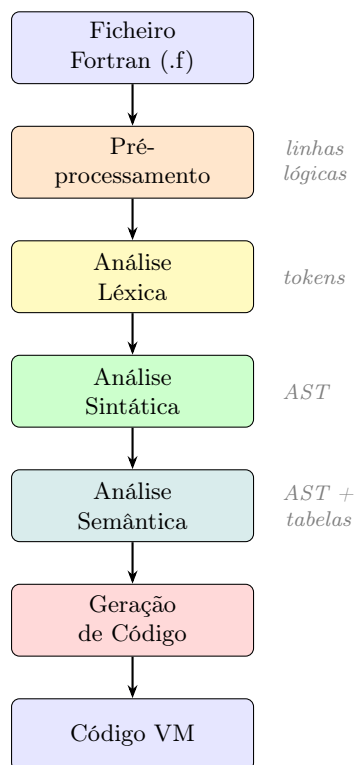


Figura 1: Pipeline de compilação.

Os módulos Python que implementam cada fase são os seguintes:



| Módulo                       | Responsabilidade                         |
|------------------------------|------------------------------------------|
| <code>lexer.py</code>        | Pré-processamento e análise léxica       |
| <code>parser.py</code>       | Análise sintática e construção da AST    |
| <code>ast_nodes.py</code>    | Definição dos nós da AST                 |
| <code>semantic.py</code>     | Análise semântica                        |
| <code>symbol_table.py</code> | Tabela de símbolos e inferência de tipos |
| <code>codegen.py</code>      | Geração de código para a VM              |
| <code>compiler.py</code>     | Fachada que compõe as fases              |
| <code>main.py</code>         | Interface de linha de comandos           |

O módulo `compiler.py` funciona como fachada: expõe as funções `run_lexer`, `run_parser`, `run_semantic` e `run_codegen` que compõem as fases necessárias de forma transparente. O `main.py` usa estas funções para implementar a interface de linha de comandos descrita na secção .

### 3. Pré-Processamento

O Fortran 77 usa um formato de colunas fixas (*fixed-form*) herdado da era dos cartões perfurados, com uma semântica de colunas rígida:

| Coluna(s) | Significado                                                            |
|-----------|------------------------------------------------------------------------|
| 1         | C, c, * ou ! iniciam uma linha de comentário completa                  |
| 2–5       | Label numérico opcional                                                |
| 6         | Carácter diferente de espaço ou 0 indica continuação da linha anterior |
| 7–72      | Código executável                                                      |
| 73+       | Ignoradas                                                              |

O pré-processamento é implementado na função `preprocess_fixed_form` do módulo `lexer.py`. Recebe o código-fonte como uma *string* e devolve uma lista de tuplos (`numero_linha_original`, `conteudo_logico`), onde cada linha lógica já incorpora as suas continuações físicas e tem os comentários removidos. A preservação do número de linha original é crucial para que as mensagens de erro refiram a linha correta no ficheiro fonte.

O processamento de cada linha física segue estes passos:

1. Truncar ao carácter 72 (colunas 73+ ignoradas);
2. Identificar e descartar linhas de comentário (coluna 1 igual a C, c, \* ou !);
3. Identificar linhas de continuação (coluna 6 não-espaço) e concatenar as colunas 7–72 à linha lógica anterior;
4. Remover comentários *inline* (marcados por ! fora de strings);
5. Converter o código para maiúsculas, preservando o conteúdo das strings.

O passo 5 merece atenção especial. Fortran 77 é *case-insensitive*, mas o lexer trabalha apenas com maiúsculas por simplicidade. A conversão é feita carácter a carácter com uma máquina de estados simples que distingue se o cursor está dentro ou fora de uma string delimitada por aspas simples. As sequências ” (aspas escapadas dentro de strings) são reconhecidas e tratadas corretamente em ambas as funções `remove_inline_comment` e `uppercase_code`. Esta abordagem centralizada garante uniformidade e evita duplicar lógica *case-insensitive* nas regras do lexer.

Após o pré-processamento, `join_logical_lines` une todas as linhas lógicas numa única *string* separadas por `\n`, que é então entregue ao lexer. Os labels numéricos das colunas 2–5 são antecidos ao conteúdo da linha de forma a serem visíveis ao lexer como `INT_LITERAL` no início da linha.



## 4. Análise Léxica

A análise léxica é implementada com `ply.lex` no módulo `lexer.py`. O lexer reconhece os seguintes tipos de tokens:

| Categoria              | Tokens                                                 |
|------------------------|--------------------------------------------------------|
| Palavras-chave         | PROGRAM, END, IF, DO, INTEGER, REAL, ... (28 no total) |
| Operadores relacionais | OP_EQ (.EQ.), OP_NE, OP_LT, OP_LE, OP_GT, OP_GE        |
| Operadores lógicos     | OP_AND, OP_OR, OP_NOT, OP_EQV, OP_NEQV                 |
| Literais lógicos       | LOGICAL_TRUE (.TRUE.), LOGICAL_FALSE (.FALSE.)         |
| Literais               | INT_LITERAL, REAL_LITERAL, STR_LITERAL, ID             |
| Operadores aritméticos | PLUS, MINUS, STAR, SLASH, DSTAR (**), DSLASH (//)      |
| Delimitadores          | LPAREN, RPAREN, COMMA, EQUALS, ...                     |
| Controlo               | NEWLINE                                                |

**Ordem das regras.** Em PLY, a ordem das funções de tokenização determina a prioridade de correspondência. Isto obrigou a cuidado especial em dois casos:

- DSTAR (\*\*) e DSLASH (//) têm de ser reconhecidos *antes* de STAR e SLASH, pois de contrário o lexer emitiria dois tokens simples em vez de um composto.
- REAL\_LITERAL tem de ser reconhecido *antes* de INT\_LITERAL, dado que qualquer número real começa por dígitos.

**Operadores pontuados.** Os operadores da forma `.XYZ.` (relacionais, lógicos e literais booleanos) são todos capturados por uma única regra e depois classificados por consulta de dicionário:

```
def t_DOT_OPERATOR(t):
    r'\.[A-Za-z]+\.'
    val = t.value.upper()
    if val in relational_ops:
        t.type = relational_ops[val]
    elif val in logical_ops:
        t.type = logical_ops[val]
    elif val in logical_literals:
        t.type = logical_literals[val]
        t.value = (val == '.TRUE.') # converte para bool Python
    else:
        print(f"[LEXER] Erro: operador desconhecido '{t.value}'", ...)
        return None
    return t
```

Esta abordagem evita duplicar expressões regulares para cada operador pontuado.

**Literais de string e o estado string.** Em Fortran 77, strings são delimitadas por aspas simples, com `''` como sequência de escape para um apóstrofo literal. Para lidar com isto sem ambiguidade, o lexer usa um *estado exclusivo* chamado `string`: ao encontrar uma aspa de abertura, transita para este estado e acumula o conteúdo num buffer global. Ao encontrar a aspa de fecho, emite o token `STR_LITERAL` e regressa ao estado inicial. No estado `string`, nenhuma regra do estado normal interfere com o conteúdo, o que é essencial para preservar os caracteres originais sem os tratar como código.

**Recuperação de erros.** Em caso de carácter ilegal, o lexer emite uma mensagem e avança um carácter (`t.lexer.skip(1)`), permitindo reportar múltiplos erros num único passe.



## 5. Análise Sintática

A análise sintática é implementada com `ply.yacc` (parser LALR(1)) no módulo `parser.py`. O parser constrói uma AST cujos nós estão definidos em `ast_nodes.py`.

### 5.1 Gramática

A gramática completa é apresentada em documento separado (`cfg.txt`). As principais produções cobrem:

- Um **ficheiro** como sequência de unidades de programa (*program\_units*);
- Uma **unidade de programa** como programa principal (com ou sem `PROGRAM nome`), uma `FUNCTION` (com ou sem tipo de retorno explícito) ou uma `SUBROUTINE` (com ou sem lista de parâmetros);
- As **declarações**, cobrindo declarações de tipo (`INTEGER X, Y(10)`), `IMPLICIT NONE` e `PARAMETER`;
- Os **statements**, cobrindo atribuição, `IF-THEN-ELSEIF-ELSE-ENDIF`, `IF` lógico de uma linha, `IF` aritmético (`IF (expr) 11,12,13`), `DO` com e sem passo, `GOTO`, `CALL` (com e sem parênteses), `RETURN` (com e sem valor), `STOP`, `PRINT` e `READ`.

A gramática permite ainda que `PROGRAM nome` seja omitido, o que o standard Fortran 77 autoriza. Esta alternativa não cria conflito `FIRST/FIRST` porque as declarações começam por *type\_spec* e a produção `declarations`  $\rightarrow \varepsilon$  permite ao parser avançar para o corpo, cujo `FIRST` é distinto de `FUNCTION/SUBROUTINE`.

**Ambiguidade FuncCall vs. Array.** Em Fortran 77, a sintaxe `F(args)` serve simultaneamente para chamar uma função e para indexar um array. O parser não consegue distingui-las sintaticamente e representa ambos os casos com o nó `FuncCall`. A distinção é adiada para a análise semântica, que consulta a tabela de símbolos para determinar se `F` é uma função ou um array.

### 5.2 Precedência de Operadores

Em vez de introduzir níveis de não-terminais na gramática (o que tornaria as regras muito mais extensas), aproveitou-se o mecanismo de declaração de precedência do PLY. A tabela seguinte mostra os operadores ordenados do menos prioritário para o mais prioritário:

| Nível | Associatividade | Operadores                               |
|-------|-----------------|------------------------------------------|
| 1     | esquerda        | .EQV. .NEQV.                             |
| 2     | esquerda        | .OR.                                     |
| 3     | esquerda        | .AND.                                    |
| 4     | direita         | .NOT.                                    |
| 5     | não-assoc       | .EQ. .NE. .LT. .LE. .GT. .GE.            |
| 6     | esquerda        | // (concatenação)                        |
| 7     | esquerda        | + -                                      |
| 8     | esquerda        | * /                                      |
| 9     | direita         | ** (potência; direita: $2^{3^2} = 2^9$ ) |
| 10    | direita         | Unário - e +                             |

Esta hierarquia corresponde exatamente à especificada pelo standard Fortran 77.



### 5.3 A AST

Todos os nós da AST herdam da classe base `Node` (em `ast_nodes.py`), que disponibiliza um campo `lineno` e o método `accept(visitor)` para o padrão *Visitor*. A raiz da AST é sempre um `ProgramFile` contendo uma lista de `Program` e/ou `Subprogram`. A hierarquia completa de nós é:

| Categoria   | Nós                                                                                                                                                                                                                                                                                                      |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Estrutura   | <code>ProgramFile</code> , <code>Program</code> , <code>Subprogram</code>                                                                                                                                                                                                                                |
| Declarações | <code>VarDecl</code> , <code>ArrayDecl</code> , <code>ImplicitNone</code> , <code>ParameterDecl</code>                                                                                                                                                                                                   |
| Statements  | <code>Assignment</code> , <code>IfThen</code> , <code>LogicalIf</code> , <code>ArithmeticIf</code><br><code>DoLoop</code> , <code>Continue</code> , <code>Goto</code> , <code>Call</code><br><code>Return</code> , <code>Stop</code> , <code>Print</code> , <code>Read</code> , <code>LabeledStmt</code> |
| Expressões  | <code>BinOp</code> , <code>UnaryOp</code> , <code>FuncCall</code> , <code>Var</code><br><code>IntLiteral</code> , <code>RealLiteral</code> , <code>StrLiteral</code> , <code>LogicalLiteral</code>                                                                                                       |

### 5.4 Resolução de DO Loops (pós-processamento)

Em Fortran 77, o corpo de um ciclo `DO` termina no statement com o label indicado (tipicamente `CONTINUE`), sem delimitador explícito. Durante o *parse*, o corpo de um `DO` não pode ser determinado imediatamente: os statements seguintes só se saberá que pertencem ao ciclo quando o label terminal for encontrado.

A solução adotada foi um pós-processamento. O parser produz uma lista plana de statements (com os `DoLoop` de corpo vazio), e `resolve_do_loops` percorre essa lista com uma pilha para agrupá-los corretamente:

```
for stmt in stmts:
    if isinstance(stmt, DoLoop):
        stack.append((stmt, []))          # abrir novo loop
    elif isinstance(stmt, LabeledStmt) and stack:
        stack[-1][1].append(stmt)        # incluir no loop corrente
        while stack and stack[-1][0].label == stmt.label:
            do_loop, body = stack.pop()   # fechar o loop
            do_loop.body = body
            if stack:
                stack[-1][1].append(do_loop)
            else:
                result.append(do_loop)
    else:
        if stack:
            stack[-1][1].append(stmt)     # corpo do loop corrente
        else:
            result.append(stmt)
```

O algoritmo suporta naturalmente ciclos aninhados (a pilha mantém um contexto por nível) e `DO` loops que partilham o mesmo label terminal. Se um `DO` loop nunca for fechado, é emitido um aviso e o loop é incluído na AST com o corpo recolhido até ao fim do âmbito, permitindo que a compilação prossiga.

**Recuperação de erros sintáticos.** A função `p_error` descarta tokens até ao próximo `NEWLINE` e reinicia o parser, permitindo reportar múltiplos erros num único passe.



## 6. Análise Semântica

A análise semântica está implementada em `semantic.py` e `symbol_table.py`. A classe `SemanticAnalyzer` percorre a AST verificando a coerência do programa e reportando diagnósticos através de objetos `Diagnostic`.

### 6.1 Tabela de Símbolos

Cada unidade de programa tem a sua própria `SymbolTable`. Existe também uma tabela global (`global_table`) que regista os subprogramas definidos no ficheiro, tornando-os visíveis em todo o programa. Após a análise, todas as tabelas locais são guardadas em `local_tables` (dicionário indexado pelo nome do âmbito) para uso posterior pelo gerador de código, que precisa de conhecer os offsets de todos os símbolos.

Cada entrada na tabela é um objeto `Symbol` com os campos:

| Campo                 | Descrição                                                                                                             |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------|
| <code>name</code>     | Nome em maiúsculas                                                                                                    |
| <code>kind</code>     | <code>variable</code> , <code>array</code> , <code>function</code> , <code>subroutine</code> , <code>parameter</code> |
| <code>type</code>     | Tipo Fortran ( <code>INTEGER</code> , <code>REAL</code> , <code>LOGICAL</code> , <code>CHARACTER</code> , ...)        |
| <code>shape</code>    | Lista de inteiros com as dimensões (só para arrays)                                                                   |
| <code>is_param</code> | <code>True</code> se for parâmetro formal de subprograma                                                              |
| <code>offset</code>   | Posição no frame de memória da VM (atribuído pelo gerador de código)                                                  |
| <code>value</code>    | Valor da constante (só para <code>PARAMETER</code> )                                                                  |

### 6.2 Passagens de Análise

A análise realiza-se em três passagens:

1. **Passagem 0** — Todos os subprogramas são registados na tabela global *antes* de qualquer análise, para permitir chamadas *forward* (uma função pode ser chamada antes de estar definida no ficheiro).
2. **Passagem 1** — Cada unidade de programa é analisada em detalhe: declarações, statements e expressões são percorridos, os símbolos registados e os erros reportados. As chamadas a subprogramas do mesmo ficheiro são acumuladas numa lista de pendentes, pois os `param_types` do subprograma chamado podem ainda não estar preenchidos.
3. **Passagem 2** — As chamadas pendentes são verificadas: o número de argumentos é comparado com o número de parâmetros formais, que a esta altura já estão todos conhecidos.

### 6.3 Verificações Implementadas

- Redecaração de variáveis e subprogramas;
- Uso de variáveis não declaradas (com suporte a *implicit typing* quando `IMPLICIT NONE` não está ativo);
- Compatibilidade de tipos em atribuições: atribuição de `LOGICAL` a numérico (ou vice-versa) é erro; atribuição de `REAL` a `INTEGER` é aviso por perda de precisão;
- Condições de `IF` e `ELSEIF` devem ser do tipo `LOGICAL`;
- Variável de controlo de `DO` recomendada como `INTEGER` (aviso se não for);
- `RETURN` fora de subprograma é erro;
- Labels de `GOTO` e de `IF` aritmético devem existir no âmbito;





- Dimensões de arrays devem ser constantes inteiras positivas;
- Número de argumentos em chamadas a subprogramas definidos no mesmo ficheiro.

## 6.4 Implicit Typing

Em Fortran 77, variáveis não declaradas recebem automaticamente um tipo: nomes começados por I–N são `INTEGER`; todos os outros são `REAL`. O compilador implementa este comportamento em `lookup_or_implicit`, que cria automaticamente uma entrada na tabela quando um símbolo desconhecido é referenciado (desde que `IMPLICIT NONE` não esteja ativo). Quando `IMPLICIT NONE` está ativo, o mesmo método lança `KeyError`, que o analisador converte em erro de compilação.

## 6.5 Inferência de Tipos

A inferência de tipos está implementada na função livre `infer_type` em `symbol_table.py`. Esta função é partilhada entre o `SemanticAnalyzer` e o `CodeGenerator`, evitando duplicação de lógica. A partilha foi possível porque `infer_type` é pura (não regista diagnósticos), cabendo ao analisador semântico acrescentar as verificações por cima.

As regras de inferência seguem o standard Fortran 77: operadores lógicos e relacionais produzem `LOGICAL`; concatenação (`//`) produz `CHARACTER`; em expressões aritméticas mistas, `REAL` prevalece sobre `INTEGER` (*widening*). As funções intrínsecas têm tipos de retorno fixos na tabela `INTRINSICS`, com suporte a funções polimórficas (e.g., `ABS` e `MAX` devolvem o tipo do argumento).

A função `eval_const` em `symbol_table.py` avalia expressões aritméticas em tempo de compilação. É usada pelo analisador semântico para verificar dimensões de arrays e pelo gerador de código para *constant folding* (secção ).

# 7. Tradução e Geração de Código

## 7.1 Ausência de Representação Intermédia

Optou-se por não implementar uma representação intermédia (IR) explícita, traduzindo diretamente da AST para código da VM. Esta decisão foi motivada por diversos motivos, mas acima de tudo por dificuldades na implementação de uma estrutura funcional e adequada para o contexto do projeto, preferindo a equipa dedicar-se a outras melhorias, como otimizações locais ao código.

## 7.2 Gestão de Frames de Memória

A VM tem dois registos de frame: `gp` (programa principal) e `fp` (subprogramas). O método `_assign_offsets` atribui offsets a todos os símbolos de um âmbito antes de gerar qualquer instrução:

- **Parâmetros formais** de subprogramas recebem offsets negativos relativamente ao `fp` ( $n$  parâmetros ficam em `fp[-n]`, ..., `fp[-1]`), pois são empilhados pelo chamador antes do `CALL`.
- **Variáveis locais** recebem offsets positivos a partir de 0, reservados com `PUSHN n`.
- **Arrays** ocupam um único slot no frame (que guarda o endereço do bloco alocado na heap com `ALLOC n`).

No programa principal, variáveis são acedidas com `PUSHG/STOREG`; nos subprogramas, com `PUSHL/STOREL`.



## 7.3 Arrays

Arrays são alocados na heap com `ALLOC n` (onde  $n$  é o produto das dimensões) durante a inicialização do âmbito. O endereço base fica guardado no slot escalar do array. Os acessos usam `LOADN/STOREN`, que recebem o endereço base e um índice flat.

O índice flat é calculado em *column-major order*, 0-based:

$$\text{flat} = (i_1 - 1) + d_1 \cdot (i_2 - 1) + d_1 d_2 \cdot (i_3 - 1) + \dots$$

## 7.4 Subprogramas

A chamada a uma `FUNCTION` ou `SUBROUTINE` segue o protocolo da VM: os argumentos são empilhados, seguidos do endereço do subprograma (`PUSHA nome`), e então é executado `CALL`. A VM guarda automaticamente `pc` e `fp` na *call stack* e restaura-os no `RETURN`.

Uma `FUNCTION` devolve o seu valor atribuindo ao símbolo com o próprio nome (e.g., `CONVRT = VAL`). No momento do `RETURN`, o gerador de código empilha esse valor (`PUSHL offset`) antes de emitir a instrução `RETURN` da VM. O valor fica no topo da stack após o retorno, onde pode ser consumido pela expressão chamadora.

## 7.5 Otimizações

Foram implementadas três otimizações diretamente no gerador de código:

**Constant Folding.** Antes de gerar qualquer instrução para uma expressão, `_gen_expr` tenta avaliá-la em tempo de compilação com `eval_const`. Se for possível, emite um único `PUSHI/PUSHF`:

```
def _gen_expr(self, node):
    if isinstance(node, (BinOp, UnaryOp)):
        val = self._eval_const(node)
        if val is not None:
            if isinstance(val, float):
                self._emit(f'PUSHF {val}')
            else:
                self._emit(f'PUSHI {int(val)}')
        return
    # ... geracao normal ...
```

Esta otimização é especialmente eficaz para índices de arrays com constantes (e.g., `A(3)` gera diretamente `PUSHI 2`) e para expressões com constantes `PARAMETER`.

**Eliminação de JUMPs Redundantes (*Peephole*).** Ao emitir um label, verifica-se se a última instrução emitida é um `JUMP` para esse mesmo label. Nesse caso, o `JUMP` é eliminado, pois é um salto para a instrução imediatamente seguinte. Isto remove sistematicamente os `JUMP` `ENDIF` que surgiriam no último ramo de um `IF-THEN-ELSE`:

```
def _emit_label(self, label):
    jump_instr = f'JUMP {label}'
    if self.output and self.output[-1] == jump_instr:
        self.output.pop() # eliminar JUMP inutil
    self.output.append(f'{label}:')
```



**Potência com Expoente Constante.** A operação `**` com expoente inteiro constante  $n > 0$  é expandida em  $n - 1$  duplicações seguidas de  $n - 1$  multiplicações, evitando um loop em *runtime*. Para expoentes negativos, gera-se `1.0 / (base**|n|)`. Para expoentes variáveis, recorre-se a um loop em *runtime* com variáveis temporárias alocadas dinamicamente no frame atual (com `PUSHN 1`), usando o contador `_next_temp_offset` para garantir que cada temporário ocupa um slot distinto.

## 8. Testes

Foram criados testes para verificar o correto funcionamento do compilador em vários cenários. Para além dos cinco exemplos do enunciado (Olá Mundo, Fatorial, Primo, Soma de Array e Conversor de Bases), foram criados testes adicionais para cobrir funcionalidades específicas e situações de erro.

As funcionalidades testadas incluem:

- Declaração e uso de variáveis escalares e arrays (uni e multidimensionais);
- Expressões aritméticas, relacionais e lógicas;
- Estruturas `IF-THEN-ELSEIF-ELSE-ENDIF` aninhadas;
- IF lógico de uma linha e IF aritmético de três labels;
- Ciclos `DO` com e sem passo, incluindo aninhamento;
- `GOTO` e labels em âmbitos com vários pontos de entrada;
- `FUNCTION` e `SUBROUTINE` com múltiplos argumentos;
- `IMPLICIT NONE` e deteção de variáveis não declaradas;
- Funções intrínsecas (`MOD`, `ABS`, `MAX`, `MIN`, `SIN`, `COS`, `INT`, `REAL`, `LEN`);
- Concatenação de strings com `//`;
- Constantes `PARAMETER` e *constant folding*;
- Recuperação de erros: um `DO` loop sem label terminal correspondente gera um aviso no `stderr` mas o compilador continua, incluindo o corpo recolhido na AST e gerando código para os statements alcançáveis.

### 8.1 Verificação das Otimizações

Para avaliar o impacto das otimizações implementadas no gerador de código, foram comparados dois programas de teste compilados com e sem otimizações ativadas. Foram analisadas duas métricas distintas:

- Número de instruções geradas;
- Número de instruções executadas pela VM.

O Teste 1 avalia sobretudo a otimização da potência com expoente constante (`**`), presentes no ficheiro `01_expr.f`, enquanto o Teste 2 avalia *constant folding* em índices de arrays e eliminação de instruções redundantes, presentes no ficheiro `07_mat2d.f`.

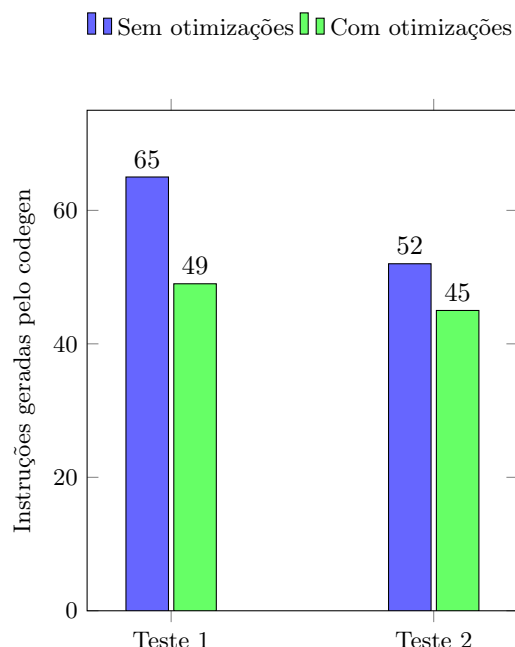


Figura 2: Número de instruções geradas pelo codegen.

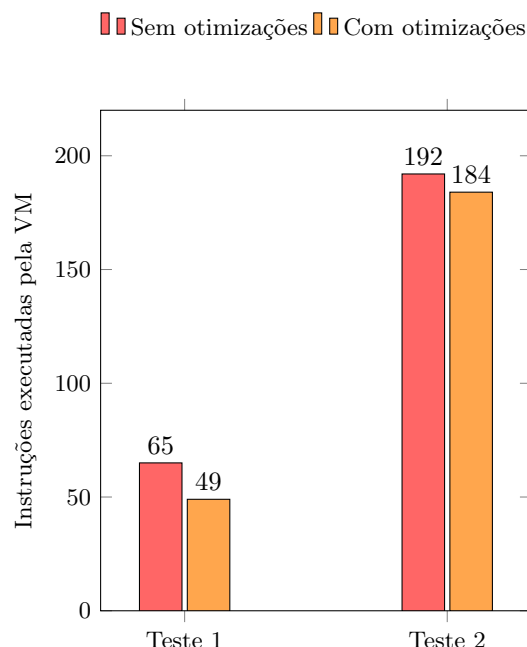


Figura 3: Número de instruções executadas pela máquina virtual.

Os resultados mostram uma redução consistente tanto no número de instruções geradas como no número de instruções efetivamente executadas pela VM. O impacto é particularmente visível no Teste 1, onde a expansão otimizada da potência constante elimina múltiplas instruções redundantes. No Teste 2, a melhoria resulta sobretudo do *constant folding* aplicado aos índices do array e da eliminação de saltos redundantes.

## 9. Manual de Utilização

### Dependências

O compilador requer Python 3.10 ou superior e algumas bibliotecas Python. Para instalar:

```
pip install -r requirements.txt
```

### Interface de Linha de Comandos

O ponto de entrada é `main.py`:

```
python main.py <fase> <alvo> [--summary] [-o SAIDA]
```

| Argumento | Descrição                                        |
|-----------|--------------------------------------------------|
| <fase>    | lex, parse, sem ou codegen                       |
| <alvo>    | Ficheiro .f ou diretoria (processa todos os .f)  |
| -summary  | Modo resumido (contagens em vez de detalhe)      |
| -o SAIDA  | Ficheiro ou diretoria de saída (só para codegen) |



## Exemplos

```
# Analise lexica
python src/main.py lex tests/programa.f

# Parse e visualizacao da AST
python src/main.py parse tests/programa.f

# Analise semantica com tabela de simbolos
python src/main.py sem tests/programa.f

# Compilacao completa, codigo para o ecrã
python src/main.py codegen tests/programa.f

# Compilar todos os .f de uma pasta, um .txt por ficheiro
python src/main.py codegen tests/ -o codigo_vm/

# Compilacao para um unico ficheiro de saida
python src/main.py codegen tests/programa.f -o saida.txt
```

Quando o alvo é uma diretoria, cada ficheiro `.f` gera um ficheiro `.txt` separado na diretoria de saída. Se o alvo de saída for um ficheiro único, os códigos são concatenados com separadores. Os erros e avisos de todas as fases são escritos em `stderr`, separados da saída principal.

## 10. Dificuldades Encontradas

**Resolução de DO Loops.** A maior dificuldade sintática foi lidar com a estrutura dos ciclos DO. Como o corpo não tem delimitador explícito, a gramática LALR(1) não consegue determinar durante o *parse* quais os statements que lhe pertencem. A solução com pilha em pós-processamento (secção 5.4) revelou-se correta mesmo para loops aninhados e para loops que partilham o mesmo label terminal.

**Protocolo de CALL na VM.** A semântica do par CALL/RETURN da VM exige que os argumentos sejam empilhados pelo chamador e que o RETURN restaure o `sp` para o valor antes dos argumentos, descartando-os automaticamente. Perceber esta semântica exigiu leitura cuidadosa da documentação da VM e testes para confirmar o comportamento em casos com subprogramas de múltiplos parâmetros.

**Ordem dos operandos em CONCAT.** A instrução CONCAT da VM concatena as duas strings na ordem em que foram empilhadas, mas a string empilhada mais recentemente é colocada *antes* no resultado. Isso obrigou a inverter a ordem de geração dos operandos do operador `//` em relação à ordem natural da expressão.

**Semântica de SUP/INF para MIN/MAX.** A documentação da VM descreve SUP como “toma  $n$  e  $m$  da pilha e empilha  $m > n$ ” — o que significa que o segundo valor a ser desempilhado é  $m$  (o empilhado mais cedo). A implementação de MAX/MIN com múltiplos argumentos via COPY 2 e SUP/INF exigiu raciocínio cuidadoso sobre esta convenção para determinar qual dos dois valores manter em cada comparação.

**Temporários para potência variável.** Quando o expoente de `**` não é conhecido em tempo de compilação, é necessário um loop em *runtime*. A VM não dispõe de instrução para aceder a posições arbitrárias da stack, pelo que foi necessário alocar slots temporários no frame atual com PUSHN 1 e acedê-los via PUSH/STOREL. O contador `_next_temp_offset` garante que os temporários não colidem com as variáveis locais declaradas.



## 11. Conclusão

O compilador desenvolvido implementa com sucesso todas as funcionalidades mínimas exigidas pelo enunciado, bem como a valorização de subprogramas (**FUNCTION** e **SUBROUTINE**). O sistema processa corretamente os cinco exemplos do enunciado e um conjunto adicional de programas de teste, gerando código executável na VM fornecida.

As decisões de design mais relevantes foram: o pré-processamento centralizado no lexer (que simplificou o tratamento do formato fixo e a conversão *case-insensitive*), o adiamento da distinção FuncCall/Array para a análise semântica, o pós-processamento com pilha para os DO loops, a partilha de **infer\_type** e **eval\_const** entre o analisador semântico e o gerador de código, e a geração direta a partir da AST sem IR intermédia.

As otimizações implementadas — *constant folding*, eliminação de JUMPs redundantes e expansão de potência com expoente constante — reduzem o tamanho e melhoram a eficiência do código gerado para os casos mais comuns, sem complexidade excessiva.

Como trabalho futuro, destacam-se: suporte a **COMMON** blocks e **EQUIVALENCE**, melhor suporte a **CHARACTER** com tamanhos declarados (**CHARACTER\*n**), implementação de uma IR para permitir otimizações globais, e suporte ao formato livre (*free-form*) das versões mais recentes do Fortran.